of chest X-ray images named the JSRT dataset, Montgomery dataset & Shenzhen dataset, which was preprocessed and augmented to create a large training set. The datasets were divided into three categories: the JSRT dataset (train set: 200, validation set: 20, test set: 20), the Montgomery dataset (train set: 110, validation set: 10, test set: 18), and Shenzhen dataset (train set: 200, validation set: 40, test set: 40). The GAN was then trained on this dataset to generate realistic lung images. The segmentation network was then trained on the synthetic lung images and evaluated on a separate test set.

The results showed that the proposed method outperformed a few other state-of-the-art methods for lung segmentation on the same dataset. The authors also conducted a detailed analysis of the performance of the method and showed that it was able to segment lungs accurately in a variety of challenging scenarios, including images with significant pathology and those with low image quality. The authors conclude that the proposed method has the potential to be a valuable tool for automatic lung segmentation in chest X-ray images and can aid in the diagnosis and monitoring of lung diseases. Table 1 summarizes the existing works.

H. RESEARCH GAPS

Thoroughly examining the literature revealed a few research problems and gaps such as class imbalance problems, generalization problems, and the challenge of computational complexity. Improvement in these areas, which can enhance our capabilities in the field of medical imaging.

1) REMOVING CLASS IMBALANCE

NIH Chest X-ray 14 dataset has 14 disease classes with 60000 samples in total and 1 non-disease class with 60000 samples. There are about 10,000 images of just the Infiltration class. Class of Hernia has the least amount of samples i.e. around 0.3% of the total samples. This indicates a huge class imbalance which leads to non-uniform results for each class. So there is a need to reduce this imbalance with augmentation by using Generative Adversarial Networks (GANs), or by merging some other datasets of the same type with it. Finally, we can evaluate the results by applying various deep learning or transfer learning models.

2) BETTER GENERALIZATION OF MODEL

Another gap that our team came up with is lower values of accuracy or F1 score or area_under_the_curve (AUC). For example, the results from research work [1] show that the same model that provides good results in some classes produces quite low results for others. One of the tasks in the future is to find such a technique that is generalized enough to produce high results for all diseases. One of the classes that have lower classification results is the Infiltration class. Although this class has a large number of samples, the results do not reflect that. One reason for suboptimal results might be the quality of the samples, which can be examined in future research.

3) IMPROVING COMPUTATIONAL COST

The NIH Chest X-ray 14 dataset comprises 112,120 samples and a size of 45 GB. The image size is 1024 by 1024 obtained from x-ray tests of around 30,000 patients. Many dense layered deep learning models struggle in the training phase Since this dataset has a large size. The computational time is so high that a single training execution of around 20 epochs costs around 5 hours. These rounded-off results were obtained from our implementation of the EfficientNet B4 model on this dataset. Such long computation times can hinder in-depth experimental research. Hence, one of the future research can be decreasing the execution complexity of models or applying some preprocessing techniques that keep the results good as well as decrease the execution time. One such way of decreasing the execution time is to use such models that have a low no of layers such as MobileNet, GoogleNet, etc.

III. RESEARCH METHOD

This section provides insights towards our methodology for the classification of lung diseases. Figure 1 provides a flowchart of the methodology followed in this research.

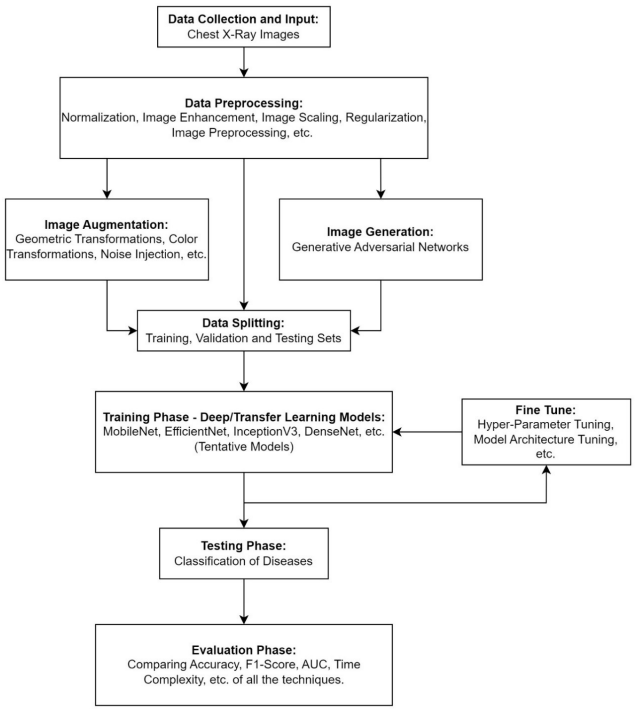

FIGURE 1. Flow chart of research methodology.

A. DATA COLLECTION AND INPUT

Human chest exams with X-rays are one of the most common and cost-effective processes for examination used by doctors. However, since X-rays have low quality as compared to other sophisticated methods such as CT scans, it is a difficult task to perform disease classification using X-ray images.

TABLE 1. Summarized review of existing works.

Ref#	Year	Technique/Models	Dataset	Train & Test Set	Results
[1]	2021	MobileNet V2 (with some additional hidden layers)	NIH ChestX-ray (ChestX-ray14) 14 class dataset and used only 64,699 samples after data augmentation	Training: 38819 (60%), Validation: 12940 (20%), Testing: 12940 (20%)	AUC: 0.81, Accuracy: 90%, Sensitivity: 45%, Specificity: 97%, F1 Score: 55%
[7]	2020	MobileNet V2 (with Multi-Kernel Depthwise Convolution (MD-Conv))	ChestX-ray14 (112,120 samples) ChestX-ray2017 (5856 samples)	-	AUC: 0.78
[8]	2018	DNet: DenseNet-121 & DNet-Loc: DenseNet-121 Location Aware	Combined 2 datasets (297,541 samples), ChestX-Ray14 (112,120), PLCO (185,421 samples), PLCO is 12 Class dataset with imbalance	-	ChestX-Ray14 – AUC: 0.84, PLCO – AUC: 0.87
[9]	2018	CheXNeXt	ChestX-ray14	-	Average AUC: 0.84
[10]	2019	ResNet-38, ResNet-50, ResNet-101, with Multi-label loss function	ChestX-ray14	Training: 70%, Validation: 10%, Testing: 20%	ResNet-38 – AUC: 0.80
[13]	2018	CNN	Subregion_demarked parenchymal-lung disease of ILD	Training set = 960 samples, Test set = 240 samples	Accuracy at CNN classifier = 95.12%
[22]	2020	Hybrid Model (VGG + spatial transformer network)	NIH chest X-ray image dataset	-	Val_acc at VDSNet = 73%, vanilla gray = 67.8%, vanilla RGB = 69%, hybrid CNN = 69.5% and VGG = 63.8%
[12]	2020	Pre-trained VGG16 + SVM Classifier & VGG16(TL) + Softmax Classifier	ICBHI 2017 Database	-	Accuracies Method 1 = 7.62%, Method 2 = 5.18%
[17]	2020	VGG16, ResNet-50, InceptionV3	Dataset from Shenzhen Hospital	Training set 80%, Validation set 10% & Test set 10%	At Segmented CXRs: VGG16 has 95%, ResNet-50 has 78%, InceptionV3 has 75% At Non-Segmented CXRs: VGG16 has 77%, ResNet-50 has 68%, InceptionV3 has 77%
[18]	2019	2D CNN as Classifier	Dataset consisting of 6 categories	Training set = 70% and test set = 30%	Accuracy = 97%
[14]	2022	Custom Net (CNN), DenseNet121, ResNet-50, Inception_V3, Vgg16	Chexpert (downsampled version-11 GB) original size 439 GB	train & validation split 80:20	DenseNet training AUROC 78 & training accuracy 87
[19]	2020	DenseNet121 visualization technique: gradCAMS	Chest X-ray 14, Total pics 112,120 from 30,805 people, Labels = 14 diseases NIH chest X-ray (due to class imbalance issue)	-	-
[15]	2021	CNN & GAN (IAGAN, DC-GAN)	ChestXRay2017 & COVIDx dataset	Dataset I (Train,Test) Normal: (-,500), Pneumonia: (33885,500), COVID_19: -, Dataset II: (67293,589 (42300,589), (0,589)	IAGAN: (Datasets I, II) Sensitivity: (0.82,0.69), Specificity: (0.84,0.69), Accuracy: (0.80,0.69)
[11]	2019	GAN (CycleGAN), DenseNet for classification	ChestXray14	Total = 112120, Labels = 14 diseases, Train Set = 70%, Validation Set = 10%, & Test Set = 20%	AUC: (Pneumonia 0.9929, Pleural-Thickening 0.9822, Fibrosis 0.9697), PR ACU (Pneumonia 0.9865, Pleural-Thickening 0.9680, Fibrosis 0.9294)
[16]	2020	GAN & U-net	JSRT dataset, Montgomery dataset & Shenzhen dataset	JSRT (train: 200, valid: 20, test: 20), Montgomery (train: 110, valid: 10, test: 18), Shenzhen (train: 200, valid: 40, test: 40)	dice-score = 0.9740, and IOU score of 0.943

Arranging an adequate number of samples for deep learning models is a difficult, if not impossible, task to do. due to patient privacy and confidentiality concerns, finding publicly available datasets is challenging. However, the National Institute of Health has made available, publicly, a dataset that is convenient to download and work with. This dataset is called NIH Chest X-ray 14 dataset, containing 112,120 samples of multiclass images along with annotated labels up to an accuracy of 90%. The labels were generated by the authors using natural language techniques. This data belongs to 30,805 patients.

The NIH CXR 14 dataset is available in the form of.png images with extension, over the internet. The size of the dataset is huge i.e., 45 GBs. Nevertheless, since it can be downloaded to the local machine and is already available on Kaggle, it makes the experiments convenient. For this research work, we used image resolution between 224 * 224 and 500 * 500, although the original is 1024 * 1024.

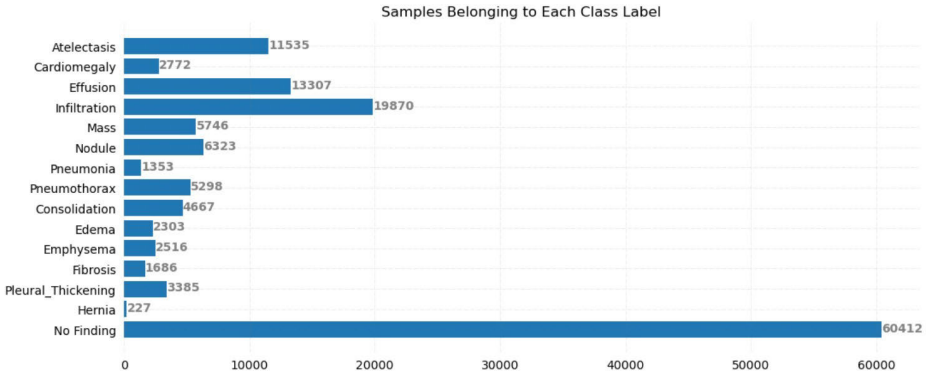

FIGURE 2. Visual class imbalance of NIH Chest Xray 14 dataset [1].

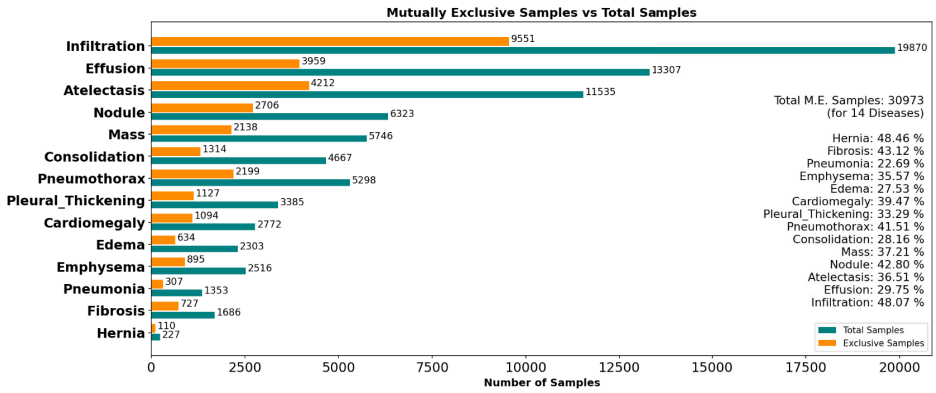

FIGURE 3. Mutually exclusive samples vs total samples.

The labels, corresponding to each image, are available in a comma-separated-value (CSV) file in the form of strings. We first converted these labels to a one-hot-encoded format and then divided the data into 3 subsets for training, validation and testing phases using Python's library named Sklearn. We used the train_test_split function with random_state equal to 0. The ratio for these divisions was 7:1:2 for 3 subsets. 80726 images were used in training, 8970 for validation and 22424 for testing the model's performance.

Figure 2 indicates the number of samples belonging to each class. It is evident how imbalanced the dataset is.

Figure 3 shows the number of samples that belong to a single class and the total samples belonging to each class. Each class has less than 50% mutually exclusive images, with the lowest as 22% for Pneumonia.

Figure 4 presents a few sample images of the dataset. All images are labelled with a disease(s) from the 14 lung pathologies or no-finding that represents the no-disease class.

B. DATA PREPROCESSING AND DATA AUGMENTATION

In data processing, data fed as input is scaled/normalized which helps the model in learning the objective function quickly and effectively. Data augmentation can be applied to a variety of data types, including images, audio, text, and time series. Some common data augmentation techniques for image data include:

- Flipping or rotating the image horizontally or vertically
- Cropping or resizing the image
- Adding noise or distortions to the image
- Changing the brightness, contrast, or color of the image
- Applying geometric transformations such as scaling, shearing, or perspective warping

1) IMAGE PREPROCESSING

It refers to the set of operations or techniques applied to raw images to prepare them for further analysis or processing. The goal of image preprocessing is to enhance or extract meaningful information from the image while reducing noise, artifacts, or other unwanted components that may interfere with subsequent analysis. The preprocessing step typically includes a series of operations such as image cropping, resizing, color correction, filtering, noise reduction, image enhancement, segmentation, and feature extraction. These operations are chosen based on the characteristics of the images and the specific application requirements.

- In this research work, the image input size was kept to either 224 * 224 or 300 * 300 which caused blurriness.
- To improve the quality, image sharpening was applied to all the dataset subsets.
- Since images are read in RGB format, we applied various rescaling techniques including multiplying each

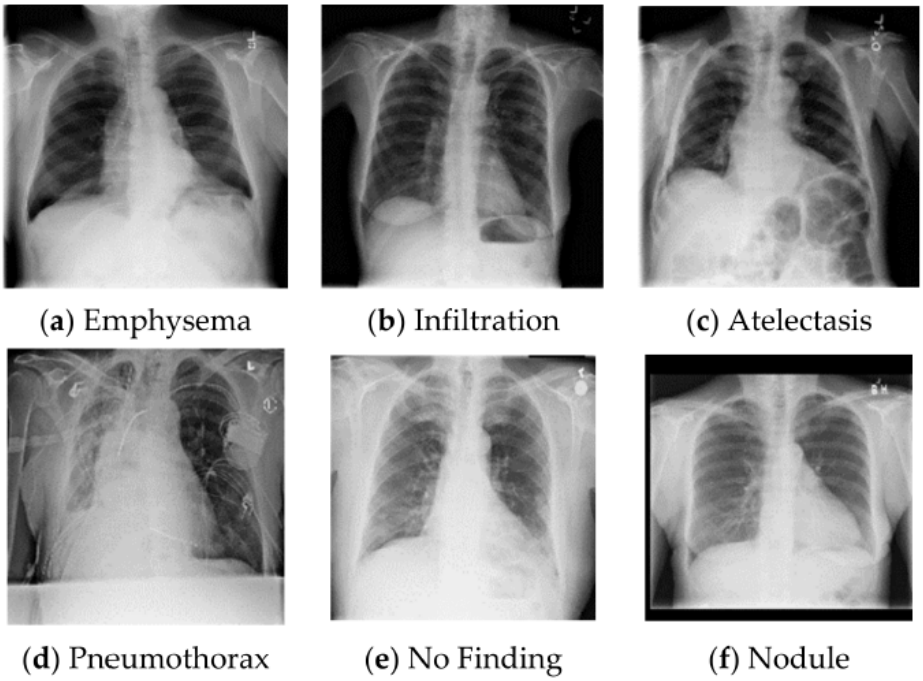

FIGURE 4. Sample images of NIH Chest X-ray 14 dataset [1].

pixel with 1/255 or converting each pixel value between −1 and 1. This not only generalizes the input but also saves memory and decreases the computational cost of operations applied. Furthermore, it also smoothens the learning of the objective function.

- The filter used for sharpening was: [0 −1 0] [−1 5 −1] [0 −1 0]

2) IMAGE AUGMENTATION

This technique is used to produce data samples from the existing samples. It can be useful when the dataset has a low number of examples or if there is a class imbalance in the dataset. We can apply various geometric methods to produce augmented data. This not only aids in learning a class imbalanced dataset but also generalizes the model's learning, hence decreasing overfitting. The model learns to handle new variants of training examples. We used the ImageDataGenerator library of Keras, to apply image augmentation.

Following are the methods that were implemented for this purpose:

- Random Rotation: It randomly rotates the input image according to the value provided. In this work, the value was set to 20 degrees.
- Sheering: It shifts the viewing angle of the image.
- Random Zoom: It randomly zooms into the input image and the value was set to 0.1 percent in this work.
- Width Shift: It shifts the image's width horizontally and its value was set to 0.1 percent of the total width of the image.
- Height Shift: It shifts the image's height vertically and its value was set to 0.1 percentage to the fraction of the image's total height.
- Horizontal Flip: This option randomly flips the image horizontally to change the sides of the image.

3) IMAGES BEFORE AND AFTER APPLYING VARIOUS PREPROCESSING AND AUGMENTATION TECHNIQUES

Figure 5 and Figure 6 present the original and processed images, respectively, from the dataset. We can see the images in smaller resolution (Figure 6) are a little blurry.

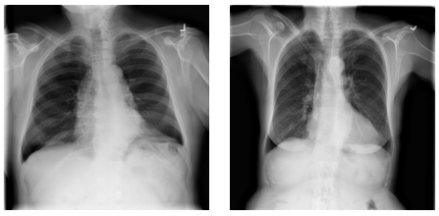

FIGURE 5. Original Images with a resolution of 1024 * 1024.

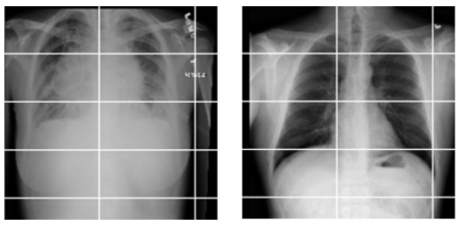

FIGURE 6. Augmented images with a resolution of 224 * 224.

Figure 7 demonstrates augmented images after image sharpening with a resolution of 224 * 224. (The left image has height shifted whereas the right image has width shifted).

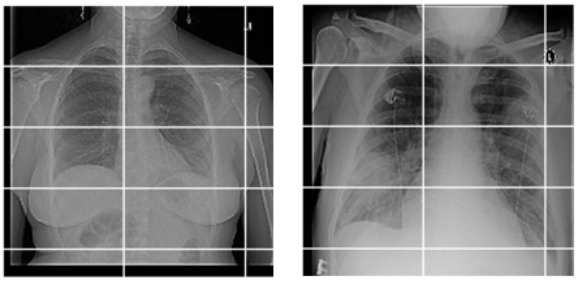

FIGURE 7. Augmented images after image sharpening with resolution of 224 * 224.

Figure 9 presents X-ray images after applying image augmentation and sharpening in RGB format; these images are fed to the network.

4) DATA GENERATION WITH GENERATIVE ADVERSARIAL NETWORK (GAN)

GAN's [30] can be used to produce data from already available samples. It can be vital for scenarios such as medical image classification wherein arranging enough data to train a large deep learning model, can be tedious. By using current available data, we can generate new samples with GAN and these synthetic samples do not belong to real patients in any direct sense. This technique has accelerated the use of deep learning in the medical field; however, it still needs a lot of improvement in terms of capturing delicate tissues from the images. In this work, we have implemented deep convolutional GAN (DCGAN) [31] to produce synthetic X-ray images containing various pathologies that are already included in the used dataset. A deep generative model with around 5 layers was implemented against a rather shallow discriminator network. The deep architecture helps in capturing minor tissues from the X-ray images. DCGAN was chosen for this study due to its balance of simplicity and effectiveness in generating high-quality images. Unlike more complex GAN variants such as WGAN, CGAN, and BEGAN, DCGAN provides a straightforward architecture that is easier to implement and optimize, making it suitable for our application in medical image generation. Additionally, DCGAN has been widely validated in various image synthesis tasks, demonstrating its robustness and efficiency in generating realistic images, which aligns well with our goal of augmenting medical datasets.

The DCGAN generates synthetic X-ray images by training a generator and a discriminator in an adversarial manner. The generator creates images from random noise, while the discriminator evaluates whether the images are real or synthetic. During training, the generator tries to produce images that are increasingly realistic, thereby 'fooling' the discriminator. This adversarial process iteratively improves the quality of the generated images. Specifically, the generator learns to map random noise vectors to image spaces that resemble the distribution of the real X-ray images, whereas the discriminator learns to distinguish between real and synthetic images. Over time, this results in the generator producing high-quality synthetic X-ray images that closely resemble real ones.

We used 3 layers in this network with 512, 256 and 128 neurons in each layer respectively. We used LeakyReLu with alpha equal to 0.2, as the activation function in all the layers except the output layer, where we used sigmoid due to binary classification. As for the generator network, it consisted of 6 layers. The number of neurons in each layer was 128, 256, 512, 512, 1024, and 1024, respectively. All the layers used LeakyReLU except the output layer, which had tanh activation function. Figure 8 shows the structure of GAN used.

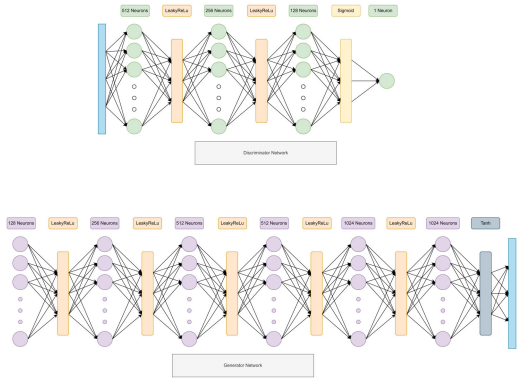

FIGURE 8. GAN architecture.

The resolution of the original images fed to the GAN was 300 by 300 with a batch size of 32. The training was done for 300k epochs for each class. It took 30 hours for each class on our local PC with a 1050Ti GPU and i7-8750H CPU. Due to time and hardware constraints, it was a challenge to train the GAN for further epochs. In Figure 10, we can see some of the samples generated for the Cardiomegaly class after 300k epochs.

Figure 11 shows individual images belonging to the class of Hernia. The images are a little pixelated, which is due to the low resolution of the latent vector and input size of the original images.

Our GAN implementation aimed to generate synthetic X-rays that closely resemble real images within the training data. This approach doesn't introduce any external data and solely augments the existing dataset to train our model.

C. TRAINING PHASE

This section highlights some of the major details involved in the training phase such as types of models, hyper-parameters, and architectures of classifiers.

1) DATASET SPLIT

The dataset split is 70% for Training, 10% for Validation and 20% for Testing.

- Training Images: 80726
- Validation Images: 8970
- Testing Images: 22424

Table 2 presents the distribution of samples belonging to each class in the training subset of the dataset.

TABLE 2. Training set sample distribution.

Class Name	Training Samples	Total Samples
Atelectasis	8276	11535
Cardiomegaly	2015	2772
Consolidation	3324	4667
Edma	1656	2303
Effusion	9717	13307
Emphysema	1758	2516
Fibrosis	1213	1686
Hernia	152	227
Infiltration	14190	19870
Mass	4105	5746
No Finding	43543	60412
Nodule	4517	6323
Pleural_Thickening	2478	3385
Pneumonia	987	1353
Pneumothorax	3784	5298

2) MODEL ARCHITECTURES

A few deep learning models are applied to the dataset to examine the performance based on different architectures. Ten models were implemented with Python's library named Keras. In terms of model initialization, we used transfer learning for the convolution part of the models and random initialization for the fully connected network. Off-the-shelf weights that were trained on the ImageNet dataset were used for quick learning of convolution network. Deep as well as shallow models were used to test which convolutional networks produce better results in feature extraction. As for the classifier network, a custom architecture was implemented. This custom architecture included 2 hidden layers and an output layer comprising 15 neuron units, i.e. a dedicated neuron for each class. 256 and 50 neurons were assigned to the first and second hidden layers, respectively. Fewer layers and neurons were used to avoid overfitting.

3) HYPER-PARAMETERS

The hyper-parameters used in the implementation were kept the same for all the models, to have a proper comparison of performance.

- Batch Size: The memory constraint of RAM and VRAM of GPU didn't allow many options to be tried in terms of batch size. Hence, the value of 32 was used for all the models trained.
- Optimizer: At first two options were tried i.e., Stochastic Gradient Descent (SGC) and Adaptive Momentum (Adam), but after some executions no significant difference was found. Therefore, we settled for the Adam optimizer for the training of models, as it is one of the advanced methods of learning the weights of the model.
- Learning Rate: Since we used already trained weights (i.e., transfer learning) for convolutional networks in our models, the learning rate was kept low. Its value was set at 0.0001.
- Activation Functions: For its simple computation and effectiveness, ReLU was applied in all the layers, except the output layer, for activation of neurons and to add non-linearity into our models. As for the output layer, sigmoid was used, since our problem is a multi-label classification problem, and each neuron has to output an independent value.
- Loss Function: Since the dataset has non-mutually exclusive labels, we need such a loss function that can consider each class independently. For this purpose, Binary_Cross_Entropy (BCS) is used to measure the loss of all the classes.

D. TESTING AND EVALUATION PHASE

In this section, we discuss the metrics that are used for the evaluation of our trained models.

1) BINARY ACCURACY

In general, accuracy states a percentage of correctly classified samples divided by all the classification results, including the correct as well as the misclassified results. Since the dataset, we worked on belongs to a multi-label classification problem, we cannot just consider the one-hot encoded labels as a single label. For example, if there are three classes in total and the sample belongs to classes 1 and 2 (label: 1 1 0). However, our model prediction states that the sample belongs to classes 1 and 3 (label: 1 0 1), so using regular accuracy would be considered incorrect. Whereas, binary accuracy calculates the percentage of labels that matched in the entire one-hot encoded label. Therefore, the binary accuracy of the above example would be 2 divided by 3, i.e. 66% accuracy.

2) RECALL

When a dataset has a class imbalance, evaluating the model based on simple accuracy is not enough. Recall is a measure of how many actual positive examples in the dataset, were classified as truly positive by the model. The recall is an important measure in the medical field since we aim to classify all the actual diseased samples as truly diseased, by the model. It is defined in [1], as follows:

$$Recall(orSensitivity) = \frac{TruePositives}{TruePositives + FalseNegatives} \tag{1}$$

3) PRECISION

Another metric that can evaluate the performance of the trained model on the imbalanced dataset is Precision. It is a measure of how many samples were actually positive from all the samples that the model predicted as positive. It is defined

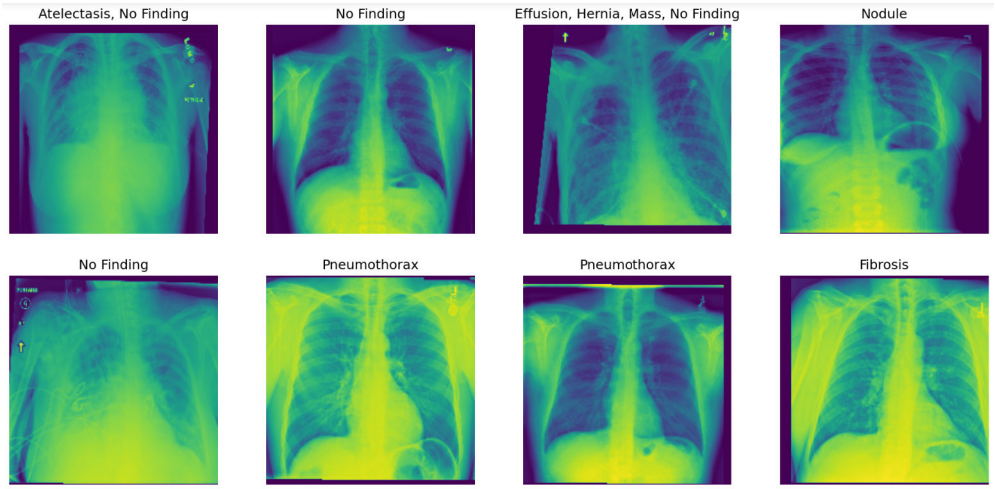

FIGURE 9. Sharpening of X-Ray images in RGB format.

FIGURE 10. Images generated after 300k epochs belonging to Cardiomegaly class.

in [1], as follows:

$$Precision = \frac{TruePositives}{TruePositives + FalsePositives} \quad (2)$$

4) SPECIFICITY

This measure can be considered as a recall but for negative samples, i.e. it gives a measure of how many actual negative examples were classified as negative by the model. It is defined in [1], as follows:

$$Specificity = \frac{TrueNegatives}{TrueNegatives + FalsePositives} \quad (3)$$

5) F1 SCORE

This measure considers both recall and precision of the model and indicates more of a balanced performance value. It is a harmonic mean of precision & recall. The F1 Score of a

FIGURE 11. Images generated by GAN belonging to Hernia class.

model will be high if more samples predicted as positive by the model were actually positive and more samples predicted as negative by the model were actually negative. It is defined in [1] with the following formula:

$$F1Score = \frac{2 * Precision * Recall}{Precision + Recall} \tag{4}$$

As the F1 Score gives equal importance to both precision & recall, it might sometimes fail to indicate the performance of the model where one is more important than the other. For example, in the medical field, we need a higher recall rate of models and can give up on the importance of precision. Therefore, F-Beta Score can be used where we assign a weight that defines the importance of each measure. A higher Beta value i.e. more than 1, gives more weightage to recall, and a lower beta value i.e. lower than 1 gives more weightage to precision.

6) AUC (ROC)

The Receiver Operating Characteristic (ROC) curve is used to examine the capability, of a model, of distinguishing between classes and is plotted using measures named True Positive Rate (TPR) and False Positive Rate (FPR). Both of these measures are directly proportional i.e. if one increases the other increases as well. The ROC curve is plotted against a number of thresholds and indicates an area_under_the_curve (AUC). The higher the area, the better the model's performance is.

TABLE 3. MobileNetV1 results without geometric image augmentation with adam optimizer.

Epochs	5	10	15
Binary Accuracy	93.2	92	92
Recall	55.2	47.7	50
Precision	52.8	42.5	46.5
F1 Score	54	45	48.2
Specificity	98.2	97	96.4
Average AUC	78.6	77.1	76

IV. EXPERIMENTAL RESULTS

In this section, the results of our experimental research and the programming environments, that were used to obtain, are documented in separate sections.

A. EXPERIMENTAL SETUP

The size of the dataset played a significant role in deciding the programming environments to conduct experiments. Deep learning models were implemented and trained on two platforms. The first one was a free GPU-based platform called "Kaggle". Its free version provides 13 Giga Bytes of RAM, a 2-core CPU, and a P100 GPU. The second platform was of our own i.e. an ASUS ROG laptop with 16 Giga Bytes of RAM, an 8th generation 6-core Intel CPU i.e. i7-8750H, and NVIDIA's GTX 1050Ti GPU. Although we had multiple options, these were not enough to explore the multi-dimensional space of experimental analysis thoroughly. Nevertheless, we obtained decent results with different models.

B. EXPERIMENTAL RESULTS (WITHOUT GEOMETRIC IMAGE AUGMENTATION)

This section presents the results of deep learning models without any geometric augmentation techniques. The original images from the dataset were passed as it is to the deep learning model. MobileNetV1 was used for this experiment. Table 3 indicates the model starts to overfit as the number of epochs is increased. The batch size was set to 16 and the input resolution of the images was 300 * 300. We scaled the pixels' values between 0 and 1 and to optimize the model we used binary cross entropy loss. To tackle this overfitting problem we use augmentation for the rest of the experiments in the research.

1) VGG16

In medical CXR data, spatial information can be very vital in the classification of diseases. Therefore, it is common to use deep learning-based networks to extract the features having important information from the images. VGG16 is one such model that has a high no of layers in its convolution network. We implemented this model with already learned weights for its feature extractor and a custom classifier with 3 layers. There were 4 executions with 5, 10, 15, and 30 epochs, having batch size 32. The model gave an above-average performance, with recall as high as 54.5 and a maximum AUC hitting mark of 67.

2) INCEPTION-V3

This model focuses on a rather optimized approach for producing good results without a huge network architecture. It has less number of learnable parameters as compared to VGG16 but produces quite good classification results. In this work, we trained it on 5, 10, and 15 epochs with a batch size of 32. It gave recall as high as 55 and AUC as high as 69, which are higher than VGG16 with a lesser number of epochs, training time, and parameters. Although the graphs and results indicate that, the model started to overfit as the epochs were increased.

3) Xception

Similar to Inception, this model also follows the idea of a shallow-depth network with optimized performance. We ran this model on 5, 10, and 15 epochs with 32 batch sizes and Adam optimizer for weight learning. It gave recall as high as